

MACs, hashes, and signatures

Article • 10/20/2022

This article discusses how message authentication codes (MACs), hashes, and signatures can be used in Universal Windows Platform (UWP) apps to detect message tampering.

Message authentication codes (MACs)

Encryption helps prevent an unauthorized individual from reading a message, but it does not prevent that individual from tampering with the message. An altered message, even if the alteration results in nothing but nonsense, can have real costs. A message authentication code (MAC) helps prevent message tampering. For example, consider the following scenario:

- Bob and Alice share a secret key and agree on a MAC function to use.
- Bob creates a message and inputs the message and the secret key into a MAC function to retrieve a MAC value.
- Bob sends the [unencrypted] message and the MAC value to Alice over a network.
- Alice uses the secret key and the message as input to the MAC function. She compares the generated MAC value to the MAC value sent by Bob. If they are the same, the message was not changed in transit.

Note that Eve, a third party eavesdropping on the conversation between Bob and Alice, cannot effectively manipulate the message. Eve does not have access to the private key and cannot, therefore, create a MAC value which would make the tampered message appear legitimate to Alice.

Creating a message authentication code ensures only that the original message was not altered and, by using a shared secret key, that the message hash was signed by someone with access to that private key.

You can use the [MacAlgorithmProvider](#) to enumerate the available MAC algorithms and generate a symmetric key. You can use static methods on the [CryptographicEngine](#) class to perform the necessary encryption that creates the MAC value.

Digital signatures are the public key equivalent of private key message authentication codes (MACs). Although MACs use private keys to enable a message recipient to verify that a message has not been altered during transmission, signatures use a private/public key pair.

This example code shows how to use the [MacAlgorithmProvider](#) class to create a hashed message authentication code (HMAC).

C#

```
using Windows.Security.Cryptography;
using Windows.Security.Cryptography.Core;
using Windows.Storage.Streams;

namespace SampleMacAlgorithmProvider
{
    sealed partial class MacAlgProviderApp : Application
    {
        public MacAlgProviderApp()
        {
            // Initialize the application.
            this.InitializeComponent();

            // Initialize the hashing process.
            String strMsg = "This is a message to be authenticated";
            String strAlgName = MacAlgorithmNames.HmacSha384;
            IBuffer buffMsg;
            CryptographicKey hmacKey;
            IBuffer buffHMAC;

            // Create a hashed message authentication code (HMAC)
            this.CreateHMAC(
                strMsg,
                strAlgName,
                out buffMsg,
                out hmacKey,
                out buffHMAC);

            // Verify the HMAC.
            this.VerifyHMAC(
                buffMsg,
                hmacKey,
                buffHMAC);
        }

        void CreateHMAC(
            String strMsg,
            String strAlgName,
            out IBuffer buffMsg,
            out CryptographicKey hmacKey,
            out IBuffer buffHMAC)
        {
            // Create a MacAlgorithmProvider object for the specified
            algorithm.
            MacAlgorithmProvider objMacProv =
            MacAlgorithmProvider.OpenAlgorithm(strAlgName);

            // Demonstrate how to retrieve the name of the algorithm used.
            String strNameUsed = objMacProv.AlgorithmName;
        }
    }
}
```

```

        // Create a buffer that contains the message to be signed.
        BinaryStringEncoding encoding = BinaryStringEncoding.Utf8;
        buffMsg = CryptographicBuffer.ConvertStringToBinary(strMsg,
encoding);

        // Create a key to be signed with the message.
        IBuffer buffKeyMaterial =
CryptographicBuffer.GenerateRandom(objMacProv.MacLength);
        hmacKey = objMacProv.CreateKey(buffKeyMaterial);

        // Sign the key and message together.
        buffHMAC = CryptographicEngine.Sign(hmacKey, buffMsg);

        // Verify that the HMAC length is correct for the selected
algorithm
        if (buffHMAC.Length != objMacProv.MacLength)
        {
            throw new Exception("Error computing digest");
        }
    }

    public void VerifyHMAC(
        IBuffer buffMsg,
        CryptographicKey hmacKey,
        IBuffer buffHMAC)
    {
        // The input key must be securely shared between the sender of
the HMAC and
        // the recipient. The recipient uses the
CryptographicEngine.VerifySignature()
        // method as follows to verify that the message has not been
altered in transit.
        Boolean IsAuthenticated =
CryptographicEngine.VerifySignature(hmacKey, buffMsg, buffHMAC);
        if (!IsAuthenticated)
        {
            throw new Exception("The message cannot be verified.");
        }
    }
}
}
}

```

Hashes

A cryptographic hash function takes an arbitrarily long block of data and returns a fixed-size bit string. Hash functions are typically used when signing data. Because most public key signature operations are computationally intensive, it is typically more efficient to sign (encrypt) a message hash than it is to sign the original message. The following procedure represents a common, albeit simplified, scenario:

- Bob and Alice share a secret key and agree on a MAC function to use.
- Bob creates a message and inputs the message and the secret key into a MAC function to retrieve a MAC value.
- Bob sends the [unencrypted] message and the MAC value to Alice over a network.
- Alice uses the secret key and the message as input to the MAC function. She compares the generated MAC value to the MAC value sent by Bob. If they are the same, the message was not changed in transit.

Note that Alice sent an unencrypted message. Only the hash was encrypted. The procedure ensures only that the original message was not altered and, by using Alice's public key, that the message hash was signed by someone with access to Alice's private key, presumably Alice.

You can use the [HashAlgorithmProvider](#) class to enumerate the available hash algorithms and create a [CryptographicHash](#) value.

Digital signatures are the public key equivalent of private key message authentication codes (MACs). Whereas MACs use private keys to enable a message recipient to verify that a message has not been altered during transmission, signatures use a private/public key pair.

The [CryptographicHash](#) object can be used to repeatedly hash different data without having to re-create the object for each use. The [Append](#) method adds new data to a buffer to be hashed. The [GetValueAndReset](#) method hashes the data and resets the object for another use. This is shown by the following example.

C#

```
public void SampleReusableHash()
{
    // Create a string that contains the name of the hashing algorithm to
    // use.
    String strAlgName = HashAlgorithmNames.Sha512;

    // Create a HashAlgorithmProvider object.
    HashAlgorithmProvider objAlgProv =
    HashAlgorithmProvider.OpenAlgorithm(strAlgName);

    // Create a CryptographicHash object. This object can be reused to
    // continually
    // hash new messages.
    CryptographicHash objHash = objAlgProv.CreateHash();

    // Hash message 1.
    String strMsg1 = "This is message 1.";
    IBuffer buffMsg1 = CryptographicBuffer.ConvertStringToBinary(strMsg1,
    BinaryStringEncoding.Utf16BE);
    objHash.Append(buffMsg1);
}
```

```

IBuffer buffHash1 = objHash.GetValueAndReset();

// Hash message 2.
String strMsg2 = "This is message 2.";
IBuffer buffMsg2 = CryptographicBuffer.ConvertStringToBinary(strMsg2,
BinaryStringEncoding.Utf16BE);
objHash.Append(buffMsg2);
IBuffer buffHash2 = objHash.GetValueAndReset();

// Hash message 3.
String strMsg3 = "This is message 3.";
IBuffer buffMsg3 = CryptographicBuffer.ConvertStringToBinary(strMsg3,
BinaryStringEncoding.Utf16BE);
objHash.Append(buffMsg3);
IBuffer buffHash3 = objHash.GetValueAndReset();

// Convert the hashes to string values (for display);
String strHash1 = CryptographicBuffer.EncodeToBase64String(buffHash1);
String strHash2 = CryptographicBuffer.EncodeToBase64String(buffHash2);
String strHash3 = CryptographicBuffer.EncodeToBase64String(buffHash3);
}

```

Digital signatures

Digital signatures are the public key equivalent of private key message authentication codes (MACs). Whereas MACs use private keys to enable a message recipient to verify that a message has not been altered during transmission, signatures use a private/public key pair.

Because most public key signature operations are computationally intensive, however, it is typically more efficient to sign (encrypt) a message hash than it is to sign the original message. The sender creates a message hash, signs it, and sends both the signature and the (unencrypted) message. The recipient calculates a hash over the message, decrypts the signature, and compares the decrypted signature to the hash value. If they match, the recipient can be fairly certain that the message did, in fact, come from the sender and was not altered during transmission.

Signing ensures only that the original message was not altered and, by using the sender's public key, that the message hash was signed by someone with access to the private key.

You can use an [AsymmetricKeyAlgorithmProvider](#) object to enumerate the available signature algorithms and generate or import a key pair. You can use static methods on the [CryptographicHash](#) class to sign a message or verify a signature.

Export restrictions on cryptography

Article • 10/20/2022

Use this info to determine if your app uses cryptography in a way that might prevent it from being listed in the Microsoft Store.

The Bureau of Industry and Security in the United States Department of Commerce regulates the export of technology that uses certain types of encryption. All apps listed in the Microsoft Store must comply with these laws and regulations because the app files can be stored in the United States. Even apps that are uploaded by app developers from other countries for distribution outside of the United States must comply with these regulations. Consequently, when submitting an app to the Microsoft Store, all app developers must make sure that their apps don't contain any technology that is restricted by these regulations.

Note The information provided here provides some guidance, but it is your responsibility as the app developer who is publishing apps in the Microsoft Store to make sure that your app complies with all applicable laws and regulations.

For more info about the U.S. Department of Commerce and the Bureau of Industry and Security, see [About the Bureau of Industry and Security](#).

For info about the Export Administration Regulations (EAR) that govern the export of technology that includes encryption, see [EAR Controls for Items That Use Encryption](#).

Governed uses

First, determine if your app uses a type of cryptography that is governed by the Export Administration Regulations. The question includes the examples shown in the list here; but remember that this list doesn't include every possible application of cryptography.

Important Consider not only the code you wrote for your app, but also all the software libraries, utilities and operating system components that your app includes or links to.

- Any use of a digital signature, such as authentication or integrity checking
- Encryption of any data or files that your app uses or accesses

- Key management, certificate management, or anything that interacts with a public key infrastructure
- Using a secure communication channel such as NTLM, Kerberos, Secure Sockets Layer (SSL), or Transport Layer Security (TLS)
- Encrypting passwords or other forms of information security
- Copy protection or digital rights management (DRM)
- Antivirus protection

For the complete and current list of cryptographic applications, see [EAR Controls for Items That Use Encryption](#) .

Non-restricted uses

Note that some of the applications of cryptography are not restricted. Here are the unrestricted tasks:

- Password encryption
- Copy protection
- Authentication
- Digital rights management
- Using digital signatures

For the complete and current list of cryptographic applications, see [EAR Controls for Items That Use Encryption](#) .

If your app calls, supports, contains, or uses cryptography or encryption for any task that is not in this list, it needs an Export Commodity Classification Number (ECCN).

If you don't have an ECCN, see [ECCN Questions and Answers](#) .

Common cryptography tasks

Article • 10/20/2022

These articles provide example code for common Universal Windows Platform (UWP) cryptography tasks, such as creating random numbers, comparing buffers, converting between strings and binary data, copying to and from byte arrays, and encoding and decoding data.

Topic	Description
Create random numbers	This example code shows how to create a random number or buffer for use in cryptography in an UWP app.
Compare buffers	This example code shows how to compare buffers in an UWP app.
Convert between strings and binary data	This example code shows how to convert between strings and binary data in an UWP app.
Copy to and from byte arrays	This example code shows how to copy to and from byte arrays in an UWP app.
Encode and decode data	This example code shows how to encode and decode base64 and hexadecimal data in an UWP app.

Create random numbers

Article • 06/24/2024

This example code shows how to create a random number or buffer for use in cryptography in a Universal Windows Platform (UWP) app.

CS

```
public string GenerateRandomData()
{
    // Define the length, in bytes, of the buffer.
    uint length = 32;

    // Generate random data and copy it to a buffer.
    IBuffer buffer = CryptographicBuffer.GenerateRandom(length);

    // Encode the buffer to a hexadecimal string (for display).
    string randomHex = CryptographicBuffer.EncodeToHexString(buffer);

    return randomHex;
}

public uint GenerateRandomNumber()
{
    // Generate a random number.
    uint random = CryptographicBuffer.GenerateRandomNumber();
    return random;
}
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Compare buffers

Article • 06/24/2024

This example code shows how to compare buffers in a Universal Windows Platform (UWP) app.

CS

```
public void CompareBuffers()
{
    // Create a hexadecimal string.
    String strHex = "30310aff";

    // Create a Base64 string that is equivalent to strHex.
    String strBase64v1 = "MDEK/w==";

    // Create a Base64 string that is not equivalent to strHex.
    String strBase64v2 = "KEDM/w==";

    // Decode strHex to a buffer.
    IBuffer buff1 = CryptographicBuffer.DecodeFromHexString(strHex);

    // Decode strBase64v1 to a buffer.
    IBuffer buff2 = CryptographicBuffer.DecodeFromBase64String(strBase64v1);

    // Decode strBase64v2 to a buffer.
    IBuffer buff3 = CryptographicBuffer.DecodeFromBase64String(strBase64v2);

    // Compare the hexadecimal-decoded buff1 to the Base64-decoded buff2.
    // The code points in the two buffers are equal, and the Boolean value
    // is true.
    Boolean bVal_1 = CryptographicBuffer.Compare(buff1, buff2);

    // Compare the hexadecimal-decoded buff1 to the Base64-decoded buff3.
    // The code points in the two buffers are not equal, and the Boolean
    value
    // is false.
    Boolean bVal_2 = CryptographicBuffer.Compare(buff1, buff3);
}
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Convert between strings and binary data

Article • 06/24/2024

This example code shows how to convert between strings and binary data in a Universal Windows Platform (UWP) app.

CS

```
public void ConvertData()
{
    // Create a string to convert.
    String strIn = "Input String";

    // Convert the string to UTF16BE binary data.
    IBuffer buffUTF16BE = CryptographicBuffer.ConvertStringToBinary(strIn,
        BinaryStringEncoding.Utf16BE);

    // Convert the string to UTF16LE binary data.
    IBuffer buffUTF16LE = CryptographicBuffer.ConvertStringToBinary(strIn,
        BinaryStringEncoding.Utf16LE);

    // Convert the string to UTF8 binary data.
    IBuffer buffUTF8 = CryptographicBuffer.ConvertStringToBinary(strIn,
        BinaryStringEncoding.Utf8);
}
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Copy to and from byte arrays

Article • 06/24/2024

This example code shows how to copy to and from byte arrays in a Universal Windows Platform (UWP) app.

CS

```
public void ByteArrayCopy()
{
    // Initialize a byte array.
    byte[] bytes = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

    // Create a buffer from the byte array.
    IBuffer buffer = CryptographicBuffer.CreateFromByteArray(bytes);

    // Encode the buffer into a hexadecimal string (for display);
    string hex = CryptographicBuffer.EncodeToHexString(buffer);

    // Copy the buffer back into a new byte array.
    byte[] newByteArray;
    CryptographicBuffer.CopyToByteArray(buffer, out newByteArray);
}
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Encode and decode data

Article • 06/24/2024

This example code shows how to encode and decode base64 and hexadecimal data in a Universal Windows Platform (UWP) app.

CS

```
public void EncodeDecodeBase64()
{
    // Define a Base64 encoded string.
    String strBase64 = "uiwyeroiugfyqcjkds897945234==";

    // Decoded the string from Base64 to binary.
    IBuffer buffer = CryptographicBuffer.DecodeFromBase64String(strBase64);

    // Encode the buffer back into a Base64 string.
    String strBase64New = CryptographicBuffer.EncodeToBase64String(buffer);
}

public void EncodeDecodeHex()
{
    // Define a hexadecimal string.
    String strHex = "30310AFF";

    // Decode a hexadecimal string to binary.
    IBuffer buffer = CryptographicBuffer.DecodeFromHexString(strHex);

    // Encode the buffer back into a hexadecimal string.
    String strHexNew = CryptographicBuffer.EncodeToHexString(buffer);
}
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Visual layer

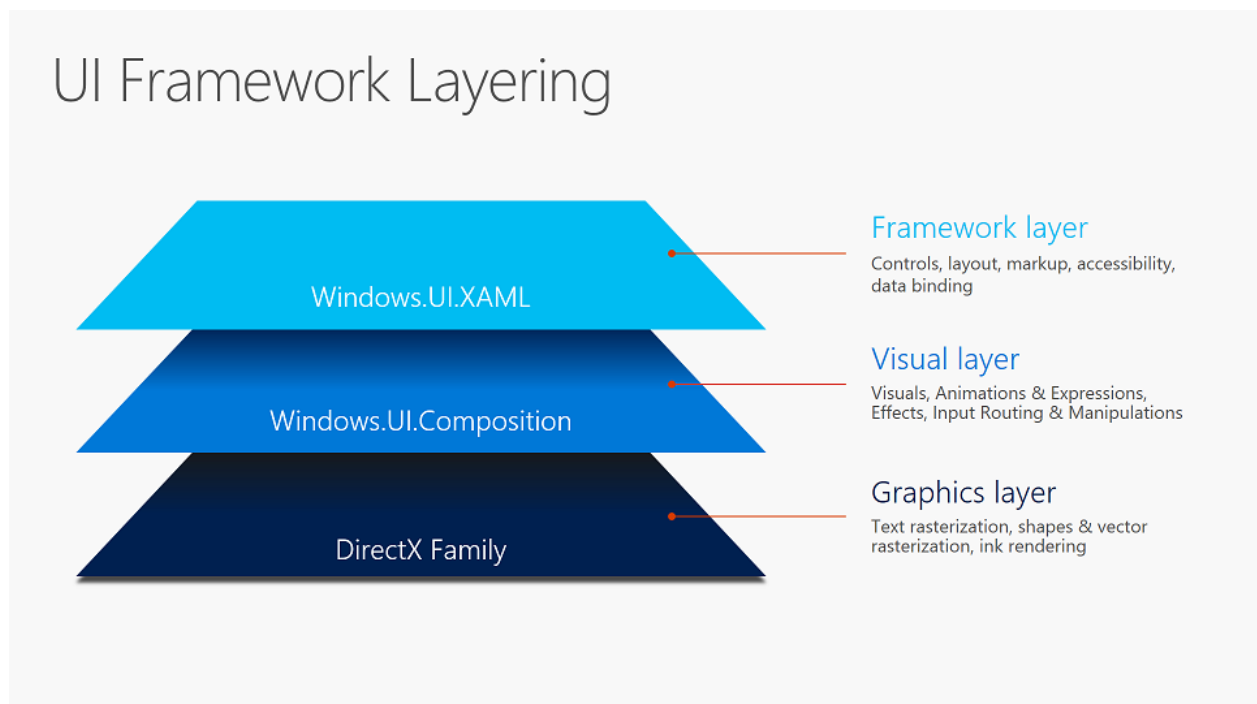
Article • 10/20/2022

The Visual layer provides a high performance, retained-mode API for graphics, effects and animations, and is the foundation for all UI across Windows devices. You define your UI in a declarative manner and the Visual layer relies on graphics hardware acceleration to ensure your content, effects and animations are rendered in a smooth, glitch-free manner independent of the app's UI thread.

Notable highlights:

- Familiar WinRT APIs
- Architected for more dynamic UI and interactions
- Concepts aligned with design tools
- Linear scalability with no sudden performance cliffs

Your Windows UWP apps are already using the Visual layer via one of the UI frameworks. You can also take advantage of Visual layer directly for custom rendering, effects and animations with very little effort.



What's in the Visual layer?

The primary functions of the Visual layer are:

1. **Content:** Lightweight compositing of custom drawn content

2. **Effects:** Real-time UI effects system whose effects can be animated, chained and customized
3. **Animations:** Expressive, framework-agnostic animations running independent of the UI thread

Content

Content is hosted, transformed and made available for use by the animation and effects system using visuals. At the base of the class hierarchy is the [Visual](#) class, a lightweight, thread-agile proxy in the app process for visual state in the compositor. Sub-classes of Visual include [ContainerVisual](#) to allow for children to create trees of visuals and [SpriteVisual](#) that contains content and can be painted with either solid colors, custom drawn content or visual effects. Together, these Visual types make up the visual tree structure for 2D UI and back most visible XAML FrameworkElements.

For more information, see the [Composition Visual](#) overview.

Effects

The Effects system in the Visual layer lets you apply a chain of filter and transparency effects to a Visual or a tree of Visuals. This is a UI effects system, not to be confused with image and media effects. Effects work in conjunction with the Animation system, allowing users to achieve smooth and dynamic animations of Effect properties, rendered independent of the UI thread. Effects in the Visual Layer provide the creative building blocks that can be combined and animated to construct tailored and interactive experiences.

In addition to animatable effect chains, the Visual Layer also supports a lighting model that allows Visuals to mimic material properties by responding to animatable lights. Visuals may also cast shadows. Lighting and shadows can be combined to create a perception of depth and realism.

For more information, see the [Composition Effects](#) overview.

Animations

The animation system in the Visual layer lets you move visuals, animate effects, and drive transformations, clips, and other properties. It is a framework agnostic system that has been designed from the ground up with performance in mind. It runs independently from the UI thread to ensure smoothness and scalability. While it lets you use familiar KeyFrame animations to drive property changes over time, it also lets

you set up mathematical relationships between different properties, including user input, letting you directly craft seamless choreographed experiences.

For more information, see the [Composition animations](#) overview.

Working with your XAML UWP app

You can get to a Visual created by the XAML framework, and backing a visible FrameworkElement, using the [ElementCompositionPreview](#) class in [Windows.UI.Xaml.Hosting](#). Note that Visuals created for you by the framework come with some limits on customization. This is because the framework is managing offsets, transforms and lifetimes. You can however create your own Visuals and attach them to an existing XAML element via ElementCompositionPreview, or by adding it to an existing ContainerVisual somewhere in the visual tree structure.

For more information, see the [Using the Visual layer with XAML](#) overview.

Working with your desktop app

You can use the Visual layer to enhance the look, feel, and functionality of your WPF, Windows Forms, and C++ Win32 desktop apps. You can migrate islands of content to use the Visual layer and keep the rest of your UI in its existing framework. This means you can make significant updates and enhancements to your application UI without needing to make extensive changes to your existing code base.

For more information, see [Modernize your desktop app using the Visual layer](#).

Additional resources

- [Full reference documentation for the API](#)
- Advanced UI and Composition samples in the [Windows Composition Samples GitHub](#) [↗](#)
- [Windows.UI.Composition Sample Gallery](#) [↗](#)
- [@windowsui Twitter feed](#) [↗](#)
- Read Kenny Kerr's MSDN Article on this API: [Graphics and Animation - Windows Composition Turns 10](#)

Composition visual

Article • 10/20/2022

Composition Visuals make up the visual tree structure which all other features of the composition API use and build on. The API allows developers to define and create one or many visual objects each representing a single node in a visual tree.

Visuals

There are several visual types that make up the visual tree structure plus a base brush class with multiple subclasses that affect the content of a visual:

- **Visual** – base object, the majority of the properties are here, and inherited by the other Visual objects.
- **ContainerVisual** – derives from **Visual**, and adds the ability to create children.
 - **SpriteVisual** – derives from **ContainerVisual**. Has the ability to associate a brush so that the Visual can render pixels including images, effects, or a solid color.
 - **LayerVisual** – derives from **ContainerVisual**. Children of the visual are flattened into a single layer.
(Introduced in Windows 10, version 1607, SDK 14393.)
 - **ShapeVisual** – derives from **ContainerVisual**. A visual tree node that is the root of a CompositionShape.
(Introduced in Windows 10, version 1803, SDK 17134.)
 - **RedirectVisual** – derives from **ContainerVisual**. The visual gets its content from another visual.
(Introduced in Windows 10, version 1809, SDK 17763.)
 - **SceneVisual** – derives from **ContainerVisual**. A container visual for the nodes of a 3D scene.
(Introduced in Windows 10, version 1903, SDK 18362.)

You can apply content and effects to SpriteVisuals using the **CompositionBrush** and its subclasses including the **CompositionColorBrush**, **CompositionSurfaceBrush** and **CompositionEffectBrush**. To learn more about brushes see **CompositionBrush Overview**.

The CompositionVisual Sample

Here, we'll look at some sample code that demonstrates the three different visual types listed previously. While this sample doesn't cover concepts like Animations or more

complex effects, it contains the building blocks that all of those systems use. (The full sample code is listed at the end of this article.)

In the sample are a number of solid color squares that can be clicked on and dragged about the screen. When a square is clicked on, it will come to the front, rotate 45 degrees, and become opaque when dragged about.

This shows a number of basic concepts for working with the API including:

- Creating a compositor
- Creating a `SpriteVisual` with a `CompositionColorBrush`
- Clipping the Visual
- Rotating the Visual
- Setting Opacity
- Changing the Visual's position in the collection.

Creating a Compositor

Creating a `Compositor` and storing it in a variable for use as a factory is a simple task. The following snippet shows creating a new `Compositor`:

C#

```
_compositor = new Compositor();
```

Creating a SpriteVisual and ColorBrush

Using the `Compositor` it's easy to create objects whenever you need them, such as a `SpriteVisual` and a `CompositionColorBrush`:

C#

```
var visual = _compositor.CreateSpriteVisual();  
visual.Brush = _compositor.CreateColorBrush(Color.FromArgb(0xFF, 0xFF, 0xFF, 0xFF));
```

While this is only a few lines of code, it demonstrates a powerful concept: `SpriteVisual` objects are the heart of the effects system. The `SpriteVisual` allows for great flexibility and interplay in color, image and effect creation. The `SpriteVisual` is a single visual type that can fill a 2D rectangle with a brush, in this case, a solid color.

Clipping a Visual

The [Compositor](#) can also be used to create clips to a [Visual](#). Below is an example from the sample of using the [InsetClip](#) to trim each side of the visual:

C#

```
var clip = _compositor.CreateInsetClip();
clip.LeftInset = 1.0f;
clip.RightInset = 1.0f;
clip.TopInset = 1.0f;
clip.BottomInset = 1.0f;
_currentVisual.Clip = clip;
```

Like other objects in the API, [InsetClip](#) can have animations applied to its properties.

Rotating a Clip

A [Visual](#) can be transformed with a rotation. Note that [RotationAngle](#) supports both radians and degrees. It defaults to radians, but it's easy to specify degrees as shown in the following snippet:

C#

```
child.RotationAngleInDegrees = 45.0f;
```

Rotation is just one example of a set of transform components provided by the API to make these tasks easier. Others include Offset, Scale, Orientation, RotationAxis, and 4x4 TransformMatrix.

Setting Opacity

Setting the opacity of a visual is a simple operation using a float value. For example, in the sample all the squares start at .8 opacity:

C#

```
visual.Opacity = 0.8f;
```

Like rotation, the [Opacity](#) property can be animated.

Changing the Visual's position in the collection

The Composition API allows for a Visual's position in a [VisualCollection](#) to be changed in a number of ways. It can be placed above another Visual with [InsertAbove](#), placed below with [InsertBelow](#), moved to the top with [InsertAtTop](#), or the bottom with [InsertAtBottom](#).

In the sample, a [Visual](#) that has been clicked is sorted to the top:

C#

```
parent.Children.InsertAtTop(_currentVisual);
```

Full Example

In the full sample, all of the concepts above are used together to construct and walk a simple tree of [Visual](#) objects to change opacity without using XAML, WWA, or DirectX. This sample shows how child [Visual](#) objects are created and added and how properties are changed.

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using System.Text;
using System.Threading.Tasks;
using Windows.ApplicationModel.Core;
using Windows.Foundation;
using Windows.UI;
using Windows.UI.Composition;
using Windows.UI.Core;

namespace compositionvisual
{
    class VisualProperties : IFrameworkView
    {
        //-----
        //
        // VisualProperties.Initialize
        //
        // This method is called during startup to associate the
        IFrameworkView with the
        // CoreApplicationView.
        //
        //-----
    }
}
```

```

-----

void IFrameworkView.Initialize(CoreApplicationView view)
{
    _view = view;
    _random = new Random();
}

//-----
//
// VisualProperties.SetWindow
//
// This method is called when the CoreApplication has created a new
CoreWindow,
// allowing the application to configure the window and start
producing content
// to display.
//
//-----

void IFrameworkView.SetWindow(CoreWindow window)
{
    _window = window;
    InitNewComposition();
    _window.PointerPressed += OnPointerPressed;
    _window.PointerMoved += OnPointerMoved;
    _window.PointerReleased += OnPointerReleased;
}

//-----
//
// VisualProperties.OnPointerPressed
//
// This method is called when the user touches the screen, taps it
with a stylus
// or clicks the mouse.
//
//-----

void OnPointerPressed(CoreWindow window, PointerEventArgs args)
{
    Point position = args.CurrentPoint.Position;

    //
    // Walk our list of visuals to determine who, if anybody, was
selected
    //
    foreach (var child in _root.Children)
    {
        //
        // Did we hit this child?
    }
}

```

```

        //
        Vector3 offset = child.Offset;
        Vector2 size = child.Size;

        if ((position.X >= offset.X) &&
            (position.X < offset.X + size.X) &&
            (position.Y >= offset.Y) &&
            (position.Y < offset.Y + size.Y))
        {
            //
            // This child was hit. Since the children are stored
back to front,
            // the last one hit is the front-most one so it wins
            //
            _currentVisual = child as ContainerVisual;
            _offsetBias = new Vector2((float)(offset.X -
position.X),
                                     (float)(offset.Y -
position.Y));
        }
    }

    //
    // If a visual was hit, bring it to the front of the Z order
    //
    if (_currentVisual != null)
    {
        ContainerVisual parent = _currentVisual.Parent as
ContainerVisual;
        parent.Children.Remove(_currentVisual);
        parent.Children.InsertAtTop(_currentVisual);
    }
}

//-----
-----
//
// VisualProperties.OnPointerMoved
//
// This method is called when the user moves their finger, stylus or
mouse with
// a button pressed over the screen.
//
//-----
-----

void OnPointerMoved(CoreWindow window, PointerEventArgs args)
{
    //
    // If a visual is selected, drag it with the pointer position
and
    // make it opaque while we drag it
    //
    if (_currentVisual != null)
    {

```

```

        //
        // Set up the properties of the visual the first time it is
        // dragged. This will last for the duration of the drag
        //
        if (!_dragging)
        {
            _currentVisual.Opacity = 1.0f;

            //
            // Transform the first child of the current visual so
that
            // the image is rotated
            //
            foreach (var child in _currentVisual.Children)
            {
                child.RotationAngleInDegrees = 45.0f;
                child.CenterPoint = new
Vector3(_currentVisual.Size.X / 2, _currentVisual.Size.Y / 2, 0);
                break;
            }

            //
            // Clip the visual to its original layout rect by using
an inset

            // clip with a one-pixel margin all around
            //
            var clip = _compositor.CreateInsetClip();
            clip.LeftInset = 1.0f;
            clip.RightInset = 1.0f;
            clip.TopInset = 1.0f;
            clip.BottomInset = 1.0f;
            _currentVisual.Clip = clip;

            _dragging = true;
        }

        Point position = args.CurrentPoint.Position;
        _currentVisual.Offset = new Vector3((float)(position.X +
_offsetBias.X),
                                           (float)(position.Y +
_offsetBias.Y),
                                           0.0f);
    }
}

//-----
-----
//
// VisualProperties.OnPointerReleased
//
// This method is called when the user lifts their finger or stylus
from the
// screen, or lifts the mouse button.
//
//-----

```

```

-----

void OnPointerReleased(CoreWindow window, PointerEventArgs args)
{
    //
    // If a visual was selected, make it transparent again when it
is
    // released and restore the transform and clip
    //
    if (_currentVisual != null)
    {
        if (_dragging)
        {
            //
            // Remove the transform from the first child
            //
            foreach (var child in _currentVisual.Children)
            {
                child.RotationAngle = 0.0f;
                child.CenterPoint = new Vector3(0.0f, 0.0f, 0.0f);
                break;
            }

            _currentVisual.Opacity = 0.8f;
            _currentVisual.Clip = null;
            _dragging = false;
        }

        _currentVisual = null;
    }
}

//-----
//
// VisualProperties.Load
//
// This method is called when a specific page is being loaded in the
// application. It is not used for this application.
//
//-----

void IFrameworkView.Load(string unused)
{
}

//-----
//
// VisualProperties.Run
//
// This method is called by CoreApplication.Run() to actually run
the

```



```

        // dispatcher's message pump.
        //
        //-----
    -----

    void IFrameworkView.Run()
    {
        _window.Activate();

        _window.Dispatcher.ProcessEvents(CoreProcessEventsOption.ProcessUntilQuit);
    }

    //-----
    -----
    //
    // VisualProperties.Uninitialize
    //
    // This method is called during shutdown to disconnect the
    CoreApplicationView,
    // and CoreWindow from the IFrameworkView.
    //
    //-----
    -----

    void IFrameworkView.Uninitialize()
    {
        _window = null;
        _view = null;
    }

    //-----
    -----
    //
    // VisualProperties.InitNewComposition
    //
    // This method is called by SetWindow(), where we initialize
    Composition after
    // the CoreWindow has been created.
    //
    //-----
    -----

    void InitNewComposition()
    {
        //
        // Set up Windows.UI.Composition Compositor, root
        ContainerVisual, and associate with
        // the CoreWindow.
        //

        _compositor = new Compositor();

        _root = _compositor.CreateContainerVisual();
    }

```

```

        _compositionTarget = _compositor.CreateTargetForCurrentView();
        _compositionTarget.Root = _root;

        //
        // Create a few visuals for our window
        //
        for (int index = 0; index < 20; index++)
        {
            _root.Children.InsertAtTop(CreateChildElement());
        }
    }

    //-----
    //
    // VisualProperties.CreateChildElement
    //
    // Creates a small sub-tree to represent a visible element in our
application.
    //
    //-----
    -----

    Visual CreateChildElement()
    {
        //
        // Each element consists of three visuals, which produce the
appearance
        // of a framed rectangle
        //
        var element = _compositor.CreateContainerVisual();
        element.Size = new Vector2(100.0f, 100.0f);

        //
        // Position this visual randomly within our window
        //
        element.Offset = new Vector3((float)(_random.NextDouble() *
400), (float)(_random.NextDouble() * 400), 0.0f);

        //
        // The outer rectangle is always white
        //
        //Note to preview API users - SpriteVisual and Color Brush
replace SolidColorBrush
        //for example instead of doing
        //var visual = _compositor.CreateSolidColorVisual() and
        //visual.Color = Color.FromArgb(0xFF, 0xFF, 0xFF, 0xFF);
        //we now use the below

        var visual = _compositor.CreateSpriteVisual();
        element.Children.InsertAtTop(visual);
        visual.Brush = _compositor.CreateColorBrush(Color.FromArgb(0xFF,
0xFF, 0xFF, 0xFF));
        visual.Size = new Vector2(100.0f, 100.0f);
    }

```

```

        //
        // The inner rectangle is inset from the outer by three pixels
all around
        //
        var child = _compositor.CreateSpriteVisual();
        visual.Children.InsertAtTop(child);
        child.Offset = new Vector3(3.0f, 3.0f, 0.0f);
        child.Size = new Vector2(94.0f, 94.0f);

        //
        // Pick a random color for every rectangle
        //
        byte red = (byte)(0xFF * (0.2f + (_random.NextDouble() /
0.8f)));
        byte green = (byte)(0xFF * (0.2f + (_random.NextDouble() /
0.8f)));
        byte blue = (byte)(0xFF * (0.2f + (_random.NextDouble() /
0.8f)));
        child.Brush = _compositor.CreateColorBrush(Color.FromArgb(0xFF,
red, green, blue));

        //
        // Make the subtree root visual partially transparent. This will
cause each visual in the subtree
        // to render partially transparent, since a visual's opacity is
multiplied with its parent's
        // opacity
        //
        element.Opacity = 0.8f;

        return element;
    }

    // CoreWindow / CoreApplicationView
    private CoreWindow _window;
    private CoreApplicationView _view;

    // Windows.UI.Composition
    private Compositor _compositor;
    private CompositionTarget _compositionTarget;
    private ContainerVisual _root;
    private ContainerVisual _currentVisual;
    private Vector2 _offsetBias;
    private bool _dragging;

    // Helpers
    private Random _random;
}

public sealed class VisualPropertiesFactory : IFrameworkViewSource
{
    //-----
    -----

```

```

        //
        // VisualPropertiesFactory.CreateView
        //
        // This method is called by CoreApplication to provide a new
IFrameworkView for
        // a CoreWindow that is being created.
        //
        //-----
-----

IFrameworkView IFrameworkViewSource.CreateView()
{
    return new VisualProperties();
}

//-----
-----

//
// main
//
//-----
-----

static int Main(string[] args)
{
    CoreApplication.Run(new VisualPropertiesFactory());

    return 0;
}
}
}

```

Composition animations

Article • 10/20/2022

The Windows.UI.Composition APIs allows you to create, animate, transform and manipulate compositor objects in a unified API layer. Composition animations provide a powerful and efficient way to run animations in your application UI. They have been designed from the ground up to ensure that your animations run at 60 FPS independent of the UI thread and to give you the flexibility to build amazing experiences using not only time, but input and other properties, to drive animations.

Motion in Windows

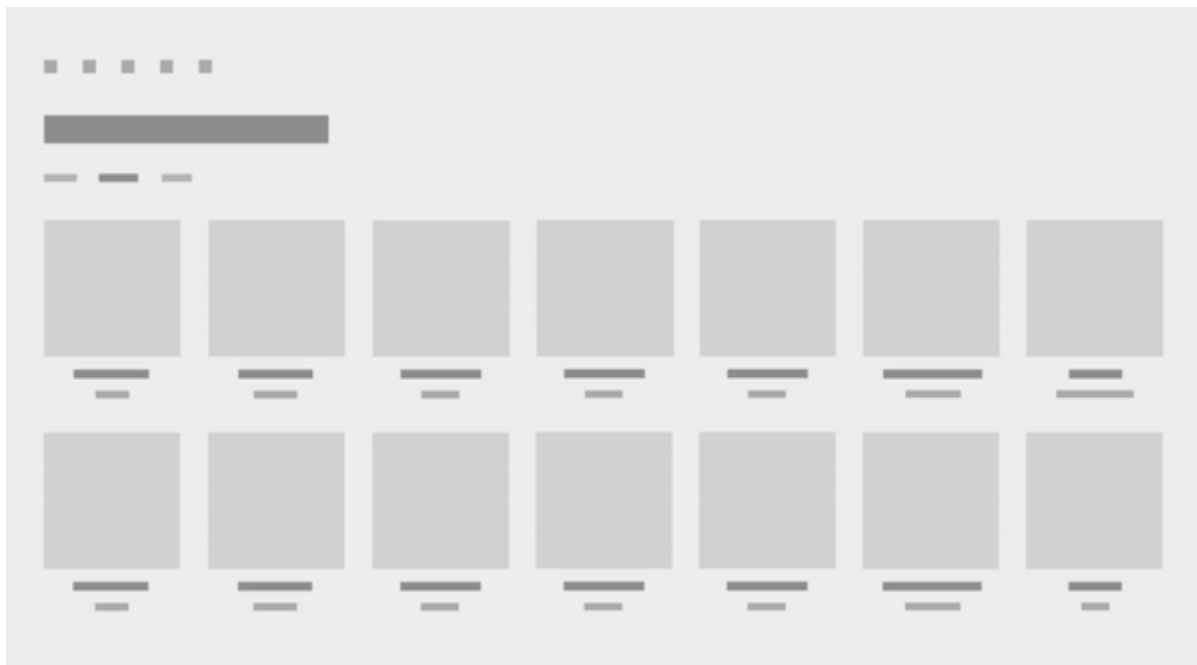
Think of motion design like a movie. Seamless transitions keep you focused on the story, and bring experiences to life. We can invite that feeling into our designs, leading people from one task to the next with cinematic ease. Motion is often the differentiating factor between a User Interface and a User Experience.

As a fundamental building block of the Windows UI Platform, CompositionAnimations provide a powerful and efficient way to create motion experiences in your application's UI. The animation engine has been designed from the ground up to ensure that your motion runs at 60 FPS, independent of the UI thread. These animations are designed to provide the flexibility to build innovative motion experiences based on time, input, and other properties.

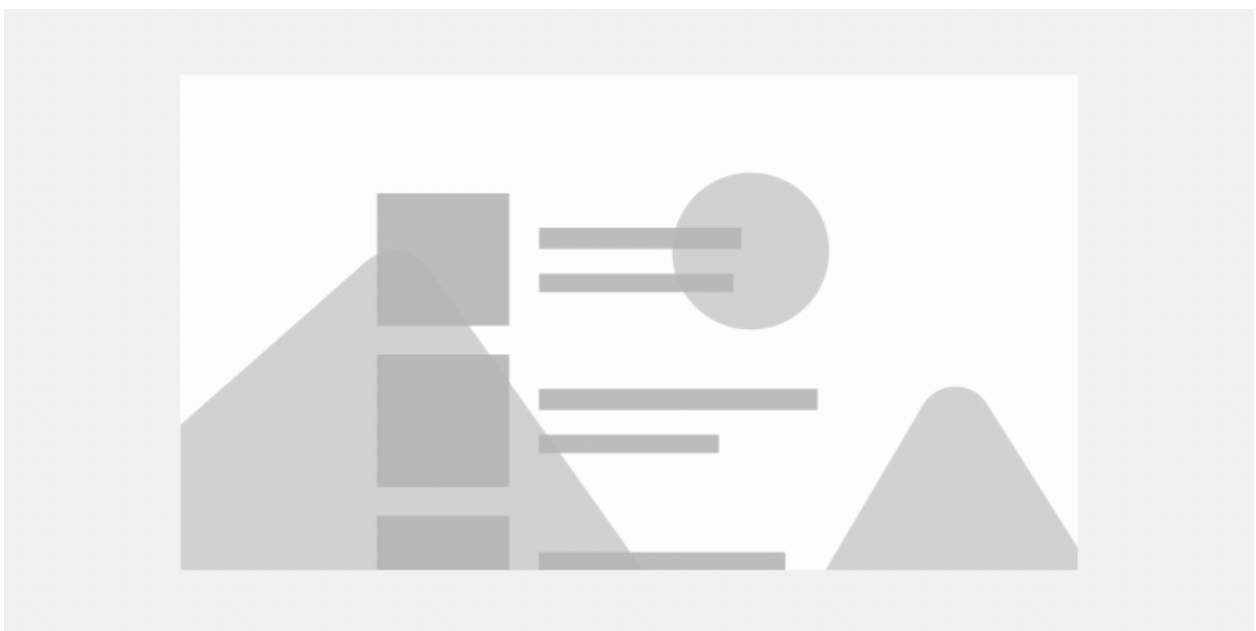
Examples of motion

Here are some examples of motion in an app.

Here, an app uses a connected animation to animate an item image as it "continues" to become part of the header of the next page. The effect helps maintain user context across the transition.



Here, a visual parallax effect moves different objects at different rates when the UI scrolls or pans to create a feeling of depth, perspective, and movement.



Using CompositionAnimations to create Motion

To generate motion in UI, developers can access animations in either XAML or the Visual Layer. Animations at the Visual Layer provide developers with a series of benefits:

- Performance – instead of the traditional UI Thread-bound animation, animations on the Windows UI platform operate on an independent thread at 60 FPS, enabling smooth motion experiences.

- **Templating Model** – animations in the Windows UI layer are templates, meaning can use a single animation on multiple objects and tweak properties or parameters without worrying of obstructing previous uses.
- **Customization** – the Windows UI layer not only makes it easy to make beautiful UI, but with a full range of animation types, possible to create new and amazing experiences with a gradient of customizations

As a developer creating experiences at the Windows UI layer, you have access to a variety of animation concepts to bring your designs to life. You can use any of these concepts to animate a property or subchannel component (when applicable) of any `CompositionObject`.

ⓘ Note

Not all properties of a `CompositionObject` are animatable. Refer to the documentation of the individual `CompositionObject` to determine whether a property is animatable.

ⓘ Note

The term *subchannel* refers to a component form of a property. For example, the X, or XY subchannel of a `Vector3 Offset` property.

Animation concept	Description
Time-based motion with KeyFrameAnimations	KeyFrameAnimations are used to directly control the entirety of a motion experience over a period of time. Developers describing a motion's start, end, interpolation in between, and duration in a traditional keyframed fashion.
Relative motion with ExpressionAnimations	ExpressionAnimations are used to describe how a motion of one object's property should be driven relative to another object's property. Developers define a mathematical equation that defines the reference-based relationship.
ImplicitAnimations	These animations are trigger-based and are defined separately from core app logic. ImplicitAnimations are used to describe how and when animations occur as a response to direct property changes.
Input-driven motion with Input Animations	Input Animations covers a set of scenarios that enable developers to describe manipulation-based motion via touch or other input modalities. These animations are driven based on active user input or gestures.

Animation concept	Description
Physics-based motion with NaturalMotionAnimations	NaturalMotionAnimations are used to describe natural and familiar motion experiences based on real-world force driven motion. Rather than defining time, developers define characteristics of the motion (for example, damping ratio for Springs)

Time based animations

Article • 10/20/2022

When a component in, or an entire user experience changes, end users often observe it in two ways: over time or instantaneously. On the Windows platform, the former is preferred over the latter - user experiences that instantly change often confuse and surprise end users because they are not able to follow what happened. The end user then perceives the experience as jarring and unnatural.

Instead, you can change your UI over time to guide the end user through, or notify them of changes to the experience. On the Windows platform, this is done by using time-based animations, also known as KeyFrameAnimations. KeyFrameAnimations let you change a UI over time and control each aspect of the animation, including how and when it starts, and how it reaches its end state. For example, animating an object to a new position over 300 milliseconds is more pleasant than instantly "teleporting" it there. When using animations instead of instantaneous changes, the net result is a more pleasant and appealing experience.

Types of time based Animations

There are two categories of time-based animations you can use to build beautiful user experiences on Windows:

Explicit Animations – as the name signifies, you explicitly start the animation to make updates. **Implicit Animations** – these animations are kicked off by the system on your behalf when a condition is met.

For this article, we will discuss how to create and use *explicit* time-based animations with KeyFrameAnimations.

For both explicit and implicit time based animations, there are different types, corresponding to the different types of properties of CompositionObjects you can animate.

- ColorKeyFrameAnimation
- QuaternionKeyFrameAnimation
- ScalarKeyFrameAnimation
- Vector2KeyFrameAnimation
- Vector3KeyFrameAnimation
- Vector4KeyFrameAnimation

Create time based animations with KeyFrameAnimations

Before describing how to create explicit time-based animations with KeyFrameAnimations, let's go over a few concepts.

- KeyFrames – These are the individual "snapshots" that an animation will animate through.
 - Defined as key & value pairs. The key represents the progress between 0 and 1, aka where in the animation lifetime this "snapshot" takes places. The other parameter represents the property value at this time.
- KeyFrameAnimation Properties – customization options you can apply to meet the needs of the UI.
 - DelayTime – time before an animation starts after StartAnimation is called.
 - Duration – duration of the animation.
 - IterationBehavior – count or infinite repeat behavior for an animation.
 - IterationCount – number of finite times a KeyFrame Animation will repeat.
 - KeyFrame Count – read of how many KeyFrames in a particular KeyFrame Animation.
 - StopBehavior – specifies the behavior of an animating property value when StopAnimation is called.
 - Direction – specifies the direction of the animation for playback.
- Animation Group – starting multiple animations at the same time.
 - Often used when wanting to animate multiple properties at the same time.

For more info, see [CompositionAnimationGroup](#).

With these concepts in mind, let's talk through the general formula for constructing a KeyFrameAnimation:

1. Identify the CompositionObject and its respective property that you need to animate.
2. Create a KeyFrameAnimation Type template of the compositor that matches the type of property you want to animate.
3. Using the animation template, start adding KeyFrames and defining properties of the animation.
 - At least one KeyFrame is required (the 100% or 1f keyframe).
 - It is recommended to define a duration as well.
4. Once you are ready to run this animation then call StartAnimation(...) on the CompositionObject, targeting the property that you want to animate. Specifically:

- `visual.StartAnimation("targetProperty", CompositionAnimation animation);`
- `visual.StartAnimationGroup(AnimationGroup animationGroup);`

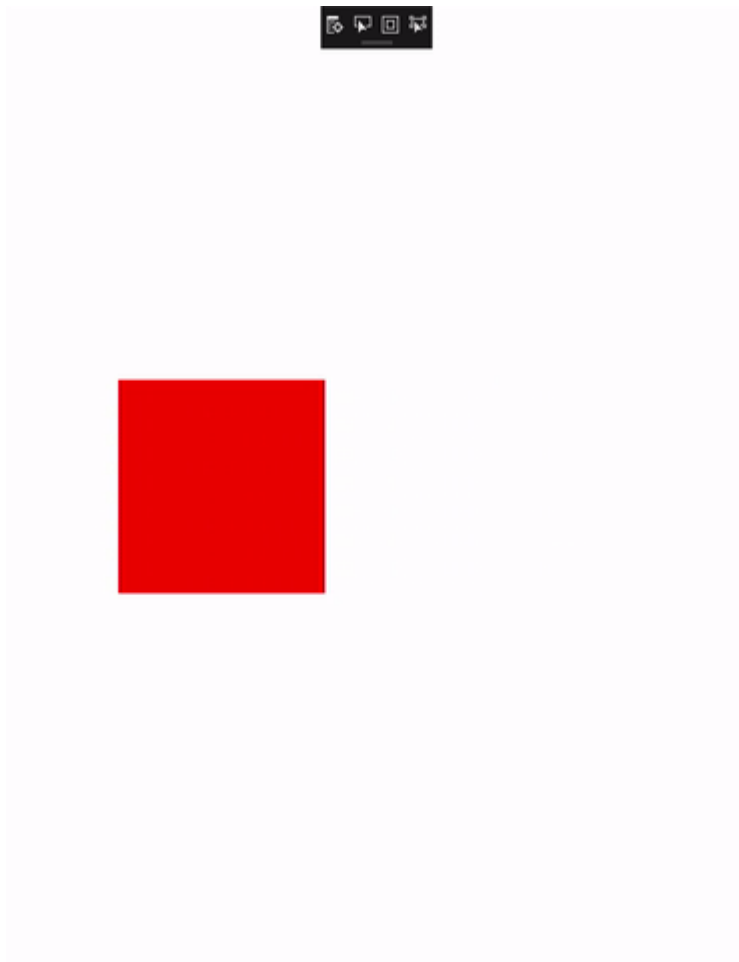
5. If you have a running animation and you want to stop the Animation or Animation Group, you can use these APIs:

- `visual.StopAnimation("targetProperty");`
- `visual.StopAnimationGroup(AnimationGroup AnimationGroup);`

Let's take a look an example to see this formula in action.

Example

In this example, you want to animate the offset of a visual from $\langle 0,0,0 \rangle$ to $\langle 200,0,0 \rangle$ over 1 second. In addition, you want to see the visual animate between these positions 10 times.



You first start by identifying the CompositionObject and property you want to animate. In this case, the red square is represented by a Composition Visual named `redVisual`. You start your animation from this object.

Next, because you want to animate the Offset property, you need to create a Vector3KeyFrameAnimation (Offset is of type Vector3). You also define the corresponding KeyFrames for the KeyFrameAnimation.

C#

```
Vector3KeyFrameAnimation animation =  
compositor.CreateVector3KeyFrameAnimation();  
animation.InsertKeyFrame(1f, new Vector3(200f, 0f, 0f));
```

Then you define the properties of the KeyFrameAnimation to describe its duration along with the behavior to animate between the two positions (current and <200,0,0>) 10 times.

C#

```
animation.Duration = TimeSpan.FromSeconds(2);  
animation.Direction =  
Windows.UI.Composition.AnimationDirection.Alternate;  
// Run animation for 10 times  
animation.IterationCount = 10;
```

Finally, in order to run an animation, you need to start it on a property of a CompositionObject.

C#

```
redVisual.StartAnimation("Offset", animation);
```

Here's the full code.

C#

```
private void AnimateSquare(Compositor compositor, SpriteVisual redVisual)  
{  
    Vector3KeyFrameAnimation animation =  
compositor.CreateVector3KeyFrameAnimation();  
    animation.InsertKeyFrame(1f, new Vector3(200f, 0f, 0f));  
    animation.Duration = TimeSpan.FromSeconds(2);  
    animation.Direction =  
Windows.UI.Composition.AnimationDirection.Alternate;  
    // Run animation for 10 times  
    animation.IterationCount = 10;  
    redVisual.StartAnimation("Offset", animation);  
}
```

Relation based animations

Article • 10/20/2022

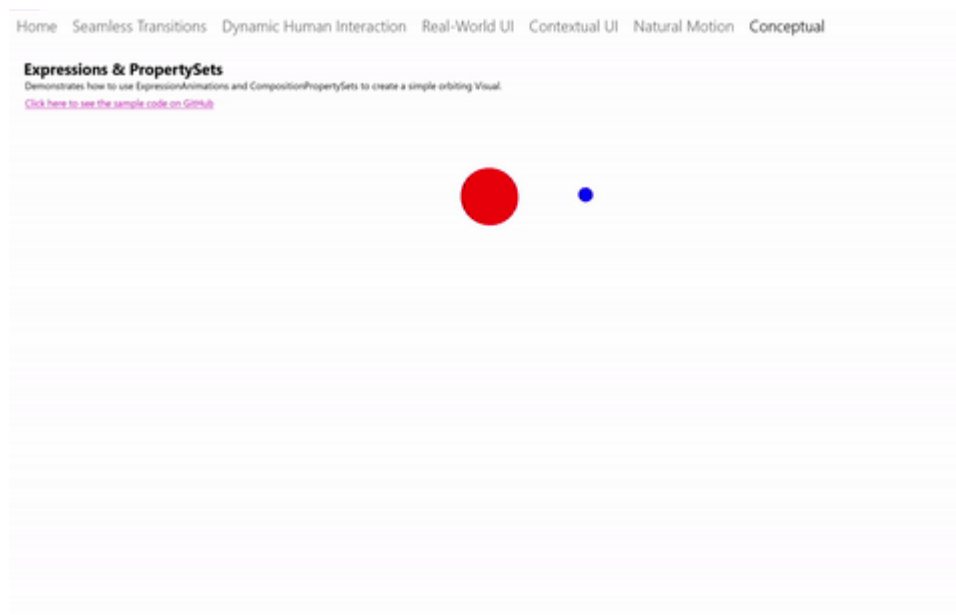
This article provides a brief overview of how to make relation-based animations using Composition ExpressionAnimations.

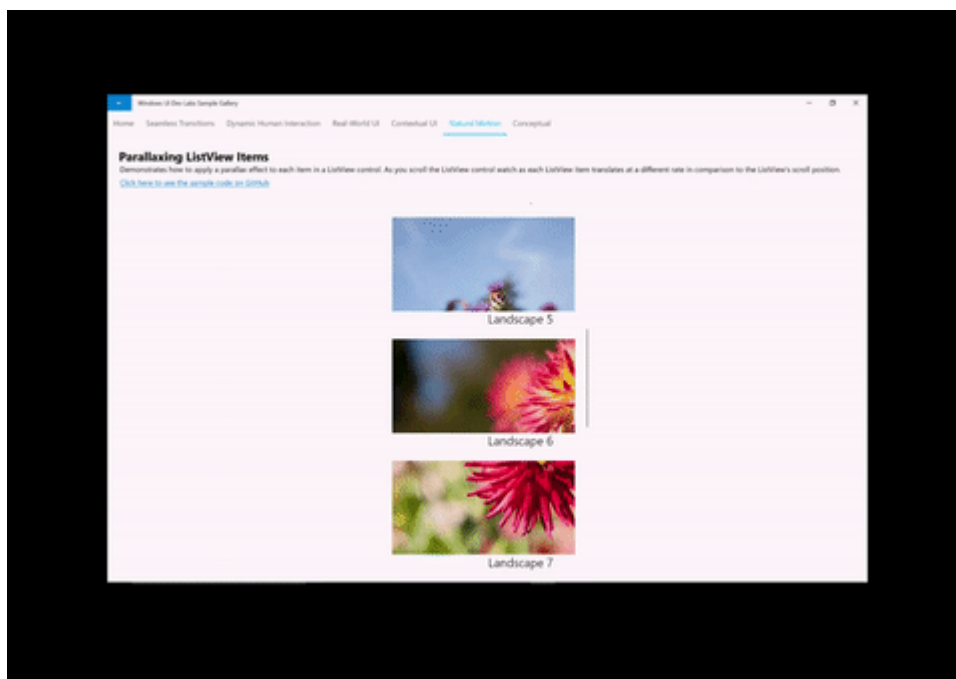
Dynamic Relation-based Experiences

When building out motion experiences in an app, there are times when the motion is not time-based, but rather dependent on a property on another object.

KeyFrameAnimations are not able to express these types of motion experiences very easily. In these specific instances, motion no longer needs to be discrete and pre-defined. Instead, the motion can dynamically adapt based on its relationship to other object properties. For example, you can animate the opacity of an object based on its horizontal position. Other examples include motion experiences like Sticky Headers and Parallax.

These types of motion experiences let you create UI that feels more connected, instead of feeling singular and independent. To the user, this gives the impression of a dynamic UI experience.





Using ExpressionAnimations

To create relation-based motion experiences, you use the ExpressionAnimation type. ExpressionAnimations (or Expressions for short), are a new type of animation that let you express a mathematical relationship – a relationship that the system uses to calculate the value of an animating property every frame. Put another way, Expressions are simply a mathematical equation that defines the desired value of an animating property per frame. Expressions are a very versatile component that can be used across a wide variety of scenarios, including:

- Relative Size, Offset animations.
- Sticky Headers, Parallax with ScrollViewer. (See [Enhance existing ScrollViewer experiences.](#))
- Snap Points with InertiaModifiers and InteractionTracker. (See [Create snap points with inertia modifiers.](#))

When working with ExpressionAnimations, there are a couple of things worth mentioning up front:

- Never Ending – unlike its KeyFrameAnimation sibling, Expressions don't have a finite duration. Because Expressions are mathematical relationships, they are animations that are constantly "running". You have the option to stop these animations if you choose.
- Running, but not always evaluating – performance is always a concern with animations that are constantly running. No need to worry though, the system is smart enough that the Expression will only re-evaluate if any of its inputs or parameters have changed.

- Resolving to the right object type – Because Expressions are mathematical relationships, it is important to make sure that the equation that defines the Expression resolves to the same type of the property being targeted by the animation. For example, if animating Offset, your Expression should resolve to a Vector3 type.

Components of an Expression

When building the mathematical relationship of an Expression, there are several core components:

- Parameters – values representing constant values or references to other Composition objects.
- Mathematical Operators – the typical mathematical operators plus(+), minus(-), multiply(*), divide(/) that join together parameters to form an equation. Also included are conditional operators such as greater than(>), equal(=), ternary operator (condition ? ifTrue : ifFalse), etc.
- Mathematical Functions – mathematical functions/shortcuts based on System.Numerics. For a full list of supported functions, see [ExpressionAnimation](#).

Expressions also support a set of keywords – special phrases that have distinct meaning only within the ExpressionAnimation system. These are listed (along with the full list of math functions) in the [ExpressionAnimation](#) documentation.

Creating Expressions with ExpressionBuilder

There are two options for building Expressions in your UWP app:

1. Build the equation as a string via the official, public API.
2. Build the equation in a type-safe object model via the ExpressionBuilder tool included with the [Windows Community Toolkit](#).

For the sake of this document, we will define our Expressions using ExpressionBuilder.

Parameters

Parameters make up the core of an Expression. There are two types of parameters:

- Constants: these are parameters representing typed System.Numeric variables. These parameters get their values assigned once when the animation is started.
- References: these are parameters representing references to CompositionObjects – these parameters continuously get their values updated after an animation is

started.

In general, References are the main aspect for how an Expression's output can dynamically change. As these references change, the output of the Expression changes as a result. If you create your Expression with Strings or use them in a templating scenario (using your Expression to target multiple CompositionObjects), you will need to name and set the values of your parameters. See the Example section for more info.

Working with KeyFrameAnimations

Expressions can also be used with KeyFrameAnimations. In these instances, you want to use an Expression to define the value of a KeyFrame at a time spot – these types KeyFrames are called ExpressionKeyFrames.

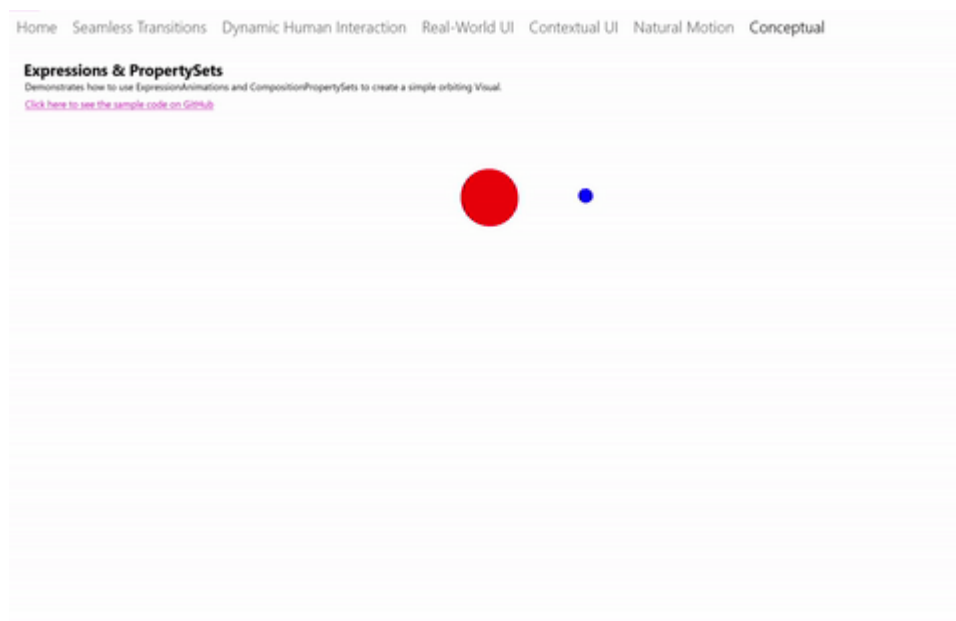
C#

```
KeyFrameAnimation.InsertExpressionKeyFrame(Single, String)  
KeyFrameAnimation.InsertExpressionKeyFrame(Single, ExpressionNode)
```

However, unlike ExpressionAnimations, ExpressionKeyFrames are evaluated only once when the KeyFrameAnimation is started. Keep in mind, you do not pass in an ExpressionAnimation as the value of the KeyFrame, rather a string (or an ExpressionNode, if you're using ExpressionBuilder).

Example

Let's now walk through an example of using Expressions, specifically the PropertySet sample from the Windows UI Sample Gallery. We'll look at the Expression managing the orbit motion behavior of the blue ball.



There are three components at play for the total experience:

1. A `KeyFrameAnimation`, animating the Y Offset of the red ball.
2. A `PropertySet` with a **Rotation** property that helps drives the orbit, animated by another `KeyFrameAnimation`.
3. An `ExpressionAnimation` that drives the Offset of the blue ball referencing the Red Ball Offset and the Rotation property to maintain a perfect orbit.

We'll be focusing on the `ExpressionAnimation` defined in #3. We will also be using the `ExpressionBuilder` classes to construct this Expression. A copy of the code used to build this experience via Strings is listed at the end.

In this equation, there are two properties you need to reference from the `PropertySet`; one is a centerpoint offset and the other is the rotation.

C#

```
var propSetCenterPoint =  
_propertySet.GetReference().GetVector3Property("CenterPointOffset");  
  
// This rotation value will animate via KFA from 0 -> 360 degrees  
var propSetRotation =  
_propertySet.GetReference().GetScalarProperty("Rotation");
```

Next, you need to define the `Vector3` component that accounts for the actual orbiting rotation.

C#

```
var orbitRotation = EF.Vector3(  
    EF.Cos(EF.ToRadians(propSetRotation)) * 150,  
    EF.Sin(EF.ToRadians(propSetRotation)) * 75, 0);
```

❗ Note

EF is a shorthand "using" notation to define ExpressionFunctions.

```
using EF =  
Microsoft.Toolkit.Uwp.UI.Animations.Expressions.ExpressionFunctions;
```

Finally, combine these components together and reference the position of the Red Ball to define the mathematical relationship.

C#

```
var orbitExpression = redSprite.GetReference().Offset + propSetCenterPoint +  
orbitRotation;  
blueSprite.StartAnimation("Offset", orbitExpression);
```

In a hypothetical situation, what if you wanted to use this same Expression but with two other Visuals, meaning 2 sets of orbiting circles. With CompositionAnimations, you can re-use the animation and target multiple CompositionObjects. The only thing you need to change when you use this Expression for the additional orbit case is the reference to the Visual. We call this templating.

In this case, you modify the Expression you built earlier. Rather than "getting" a reference to the CompositionObject, you create a reference with a name and then assign different values:

C#

```
var orbitExpression =  
ExpressionValues.Reference.CreateVisualReference("orbitRoundVisual");  
orbitExpression.SetReferenceParameter("orbitRoundVisual", redSprite);  
blueSprite.StartAnimation("Offset", orbitExpression);  
// Later on ... use same Expression to assign to another orbiting Visual  
orbitExpression.SetReferenceParameter("orbitRoundVisual", yellowSprite);  
greenSprite.StartAnimation("Offset", orbitExpression);
```

Here is the code if you defined your Expression with Strings via the public API.

C#

```
ExpressionAnimation expressionAnimation =  
compositor.CreateExpressionAnimation("visual.Offset + " +  
"propertySet.CenterPointOffset + " +  
"Vector3(cos(ToRadians(propertySet.Rotation)) * 150," +
```

```
"sin(ToRadians(propertySet.Rotation)) * 75, 0)");
```

```
var propSetCenterPoint =  
_propertySet.GetReference().GetVector3Property("CenterPointOffset");  
var propSetRotation =  
_propertySet.GetReference().GetScalarProperty("Rotation");  
expressionAnimation.SetReferenceParameter("propertySet", _propertySet);  
expressionAnimation.SetReferenceParameter("visual", redSprite);
```

Natural motion animations

Article • 10/20/2022

This article provides a brief overview of the NaturalMotionAnimation space and how to conceptually think about using these types of animations in your UI.

Making motion feel familiar and natural

Great apps are ones that create experiences that capture and retain user attention, and help guide users through tasks. Motion is the key differentiating factor that separates a User Interface from a User Experience – eliciting a connection between users and the application they are interacting with. The better the connection, the higher engagement and satisfaction from end users.

One way motion can help build this connection is by creating experiences that look and feel familiar to users. Users have an unconscious expectation for how they perceive motion that is based on real life experiences. We see how objects slide across the floor, fall off the table, bounce into one another and oscillate with a spring. Motion that leverages this expectation by being based on real-world physics looks and feels more natural in our eyes. The motion becomes more natural and interactive, but more importantly, the entire experience becomes more memorable and delightful.



Motion without Animation



Motion with Cubic Bezier



The net result is higher user engagement and retention with the app.

Balancing Control and Dynamism

In traditional UI, [KeyFrameAnimations](#) are the predominant way to describe motion. KeyFrames provided the utmost control to designers and developers to define the start, end, and interpolation. Although this is very useful in many cases, KeyFrame Animations are not very dynamic; the motion is not adaptive and looks the same under any condition.

On the other end of the spectrum, there are simulations often seen in gaming and physics engines. These experiences often are the most life-like and dynamic that users interact with – creating that sense of ambiance and randomness that users see every day. Although this makes motion feel more alive and dynamic, designers and developers have less control, therefore making it more difficult to integrate into traditional UI.



[NaturalMotionAnimations](#) exist to help bridge this divide – enabling a balance of control for the important elements of an animation like start/finish, but maintaining motion that looks and feels natural and dynamic.

ⓘ Note

NaturalMotionAnimations are not meant as a replacement for KeyFrame Animations – there are still places in the Fluent design language where KeyFrames are recommended. NaturalMotionAnimations are meant to be used in places where motion is required but KeyFrame Animations are not dynamic enough.

Using NaturalMotionAnimations

Starting with the Fall Creators Update, you have access to a new motion experience: **spring animations**. See [Spring animations](#) for a more in depth walkthrough of springs.

This motion type is achieved by using the new NaturalMotionAnimation – a new animation type centered on enabling developers to build more familiar and natural feeling motion into their UI, with a balance of control and dynamism. They expose the following capabilities:

- Define the start and end values.
- Define InitialVelocity and tie to input with InteractionTracker.
- Define motion specific properties (such as DampingRatio for springs.)

General Formula to get started:

1. Create the NaturalMotionAnimation off the Compositor using one of the **Create** methods.
2. Define the properties of the animation.
3. Pass the NaturalMotionAnimation as a parameter to the StartAnimation call off of a CompositionObject.

- Or set to the Motion property of an InteractionTracker InertiaModifier.

A basic example using a spring NaturalMotionAnimation to make a Visual "spring" to a new X Offset location:

C#

```
_springAnimation = _compositor.CreateSpringScalarAnimation();
_springAnimation.Period = TimeSpan.FromSeconds(0.07);
_springAnimation.DelayTime = TimeSpan.FromSeconds(1);
_springAnimation.EndPoint = 500f;
sectionNav.StartAnimation("Offset.X", _springAnimation);
```

Spring animations

Article • 10/20/2022

The article shows how to use spring `NaturalMotionAnimations`.

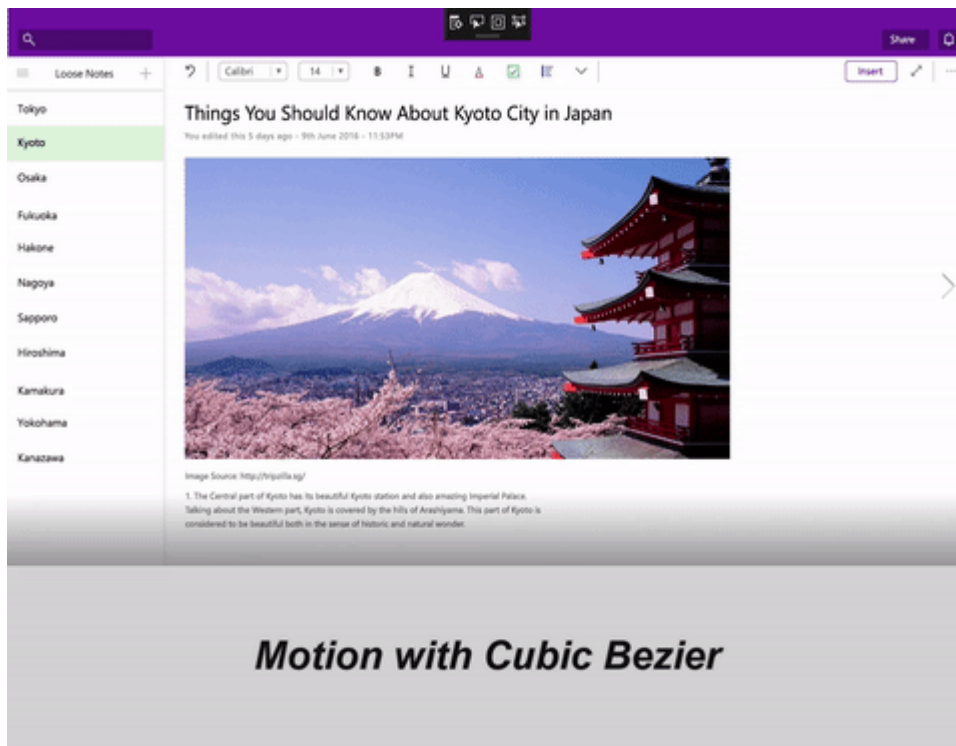
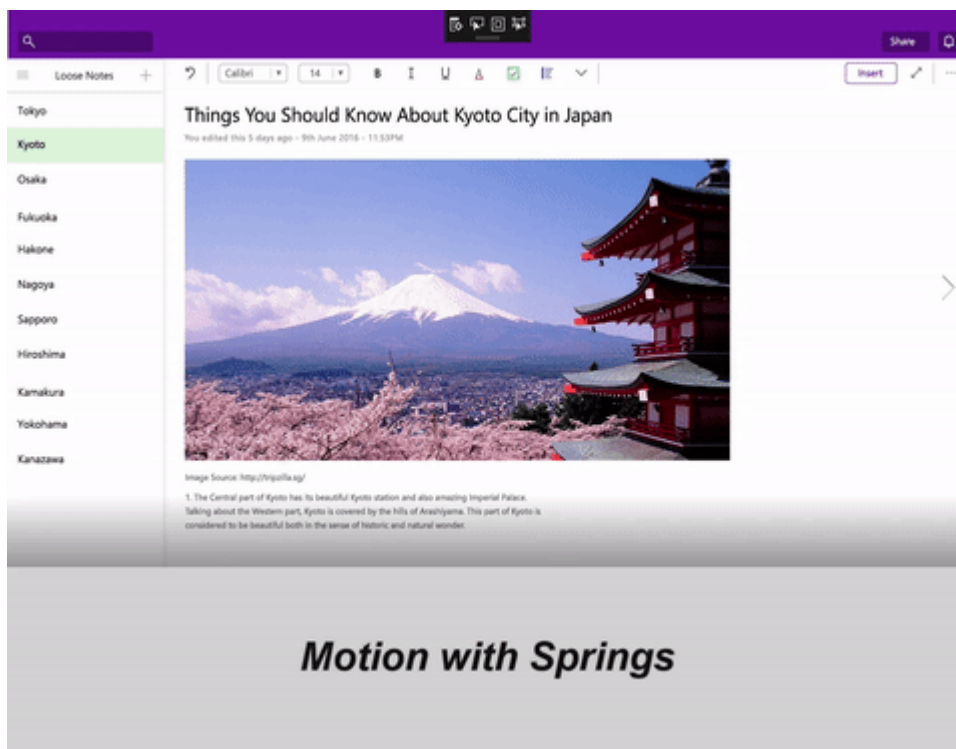
Prerequisites

Here, we assume that you're familiar with the concepts discussed in these articles:

- [Natural motion animations](#)

Why springs?

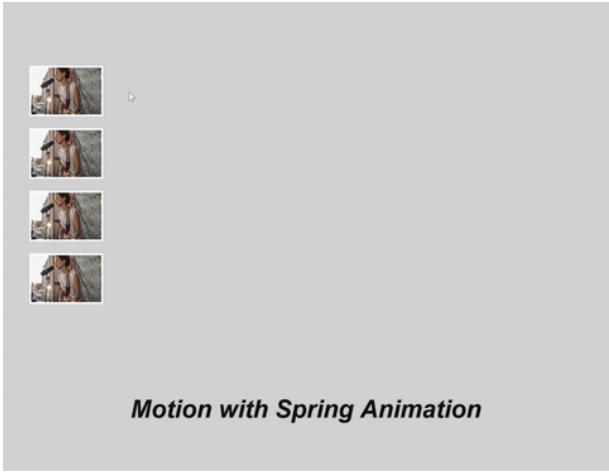
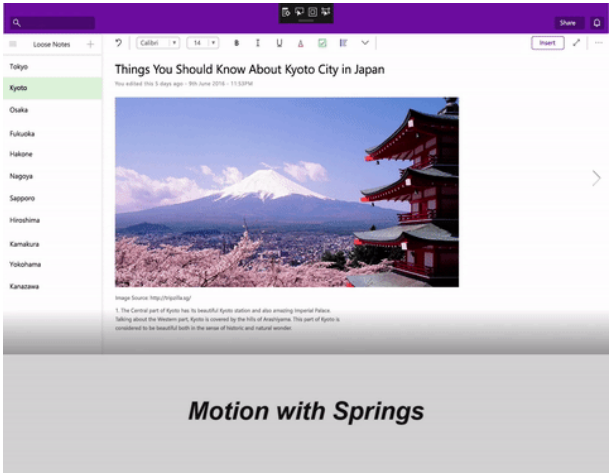
Springs are a common motion experience we've all experienced at some point in our lives; ranging from slinky toys to Physics classroom experiences with a spring-tied block. The oscillating motion of a spring often incites a playful and lighthearted emotional response from those who observe it. As a result, the motion of a spring translates well into application UI for those who are looking to create a livelier motion experience that "pops" more to the end user than a traditional Cubic Bezier. In these cases, spring motion not only creates a livelier motion experience, but also can help draw attention to new or currently animating content. Depending on the application branding or motion language, the oscillation is more pronounced and visible, but in other cases it is more subtle.



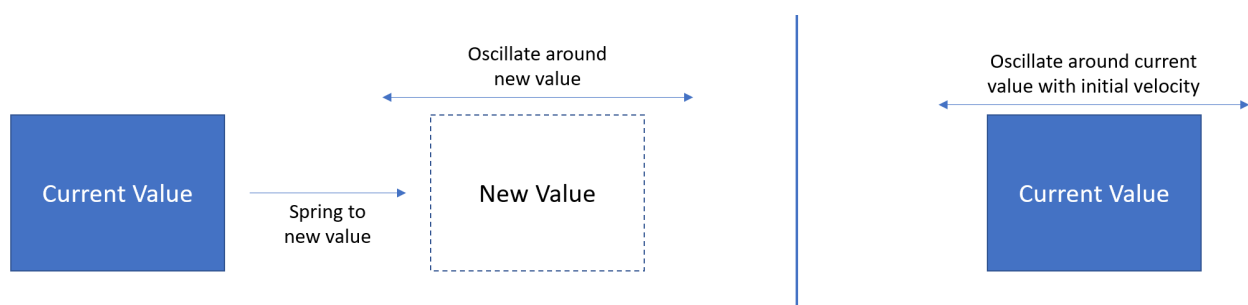
Using springs in your UI

As mentioned previously, springs can be a useful motion to integrate into your app to introduce a very familiar and playful UI experience. Common usage of springs in UI are:

Spring Usage Description	Visual Example
--------------------------	----------------

Spring Usage Description	Visual Example
Making a motion experience "pop" and look livelier. (Animating Scale)	
Making a motion experience subtly feel more energetic (Animating Offset)	

In each of these cases, the spring motion can be triggered either by "springing to" and oscillating around a new value or oscillating around its current value with some initial velocity.



Defining your spring motion

You create a spring experience by using the `NaturalMotionAnimation` APIs. Specifically, you create a `SpringNaturalMotionAnimation` by using the `Create*` methods off the `Compositor`. You are then able to define the following properties of the motion:

- **DampingRatio** – expresses the level of damping of the spring motion used in the animation.

Damping Ratio Value	Description
DampingRatio = 0	Undamped – the spring will oscillate for a long time
0 < DampingRatio < 1	Underdamped – spring will oscillate from a little to a lot.
DampingRatio = 1	Criticallydamped – the spring will perform no oscillation.
DampingRatio > 1	Overdamped – the spring will quickly reach its destination with an abrupt deceleration and no oscillation

- Period – the time it takes the spring to perform a single oscillation.
- Final / Starting Value – defined starting and ending positions of the spring motion (if not defined, starting value and/or final value will be current value).
- Initial Velocity – programmatic initial velocity for the motion.

You can also define a set of properties of the motion that are the same as `KeyFrameAnimations`:

- DelayTime / Delay Behavior
- StopBehavior

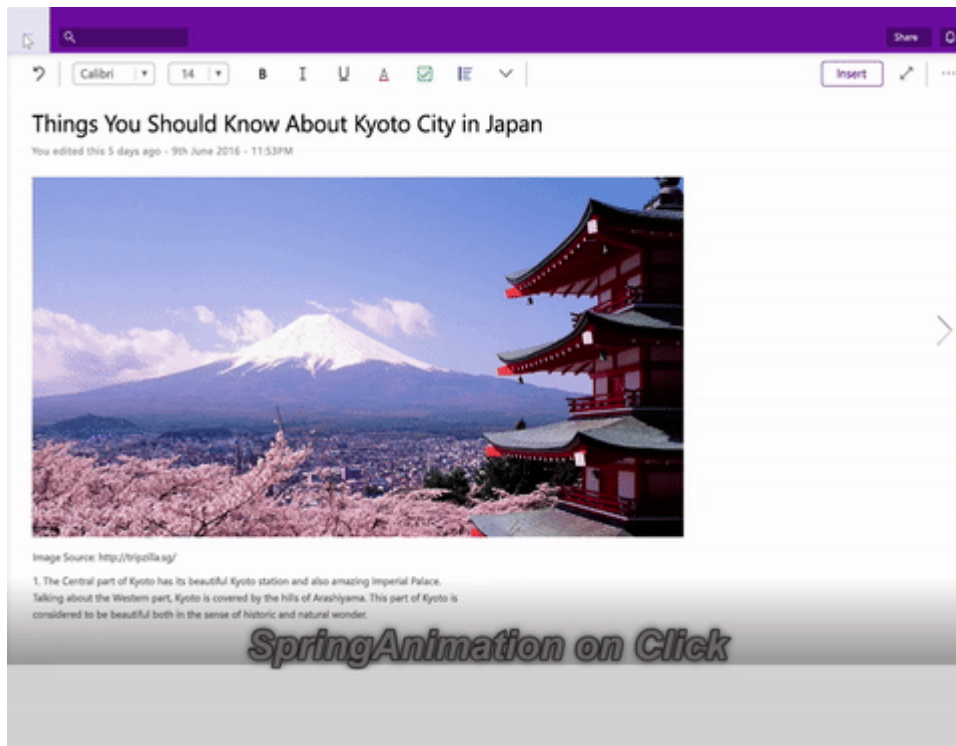
In the common cases of animating Offset and Scale/Size, the following values are recommended by the Windows Design team for DampingRatio and Period for different types of springs:

Property	Normal Spring	Dampened Spring	Less-Dampened Spring
Offset	Damping Ratio = 0.8 Period = 50 ms	Damping Ratio = 0.85 Period = 50 ms	Damping Ratio = 0.65 Period = 60 ms
Scale/Size	Damping Ratio = 0.7 Period = 50 ms	Damping Ratio = 0.8 Period = 50 ms	Damping Ratio = 0.6 Period = 60 ms

Once you have defined the properties, you can then pass in your spring `NaturalMotionAnimation` into the `StartAnimation` method of a `CompositionObject` or the `Motion` property of an `InteractionTracker InertiaModifier`.

Example

In this example, you create a navigation and canvas UI experience in which, when the user clicks an expand button, a navigation pane is animated out with a springy, oscillation motion.



Start by defining the spring animation within the clicked event for when the navigation pane appears. You then define the properties of the animation, using the `InitialValueExpression` feature to use an Expression to define the `FinalValue`. You also keep track of whether the pane is opened or not and, when ready, start the animation.

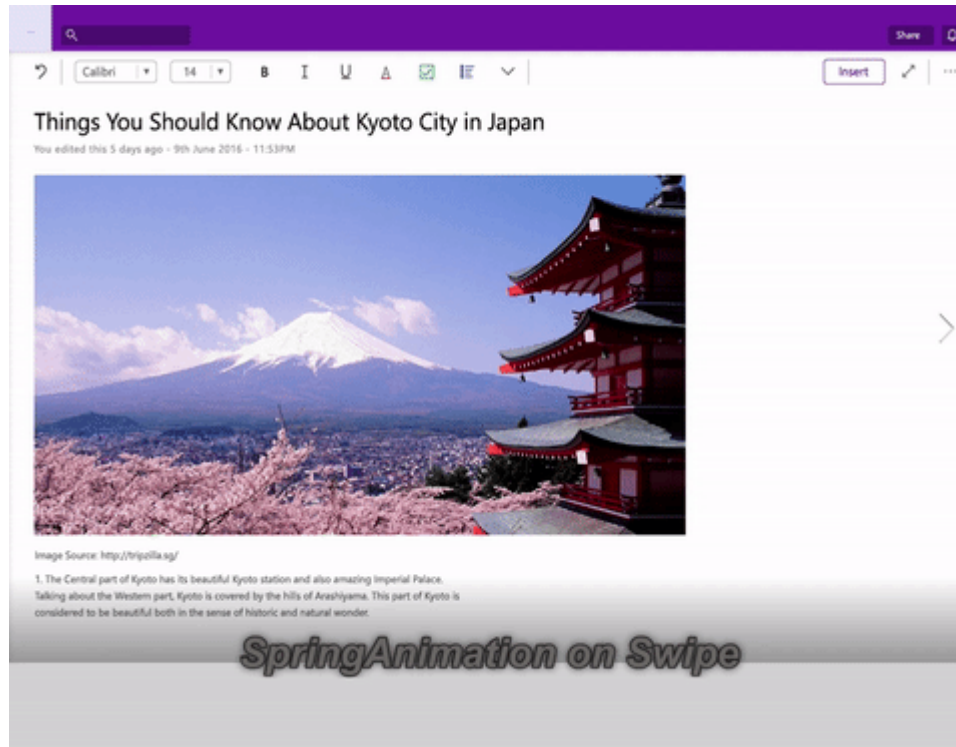
C#

```
private void Button_Clicked(object sender, RoutedEventArgs e)
{
    _springAnimation = _compositor.CreateSpringScalarAnimation();
    _springAnimation.DampingRatio = 0.75f;
    _springAnimation.Period = TimeSpan.FromSeconds(0.5);

    if (!_expanded)
    {
        _expanded = true;
        _propSet.InsertBoolean("expanded", true);
        _springAnimation.InitialValueExpression["FinalValue"] = "this.StartingValue
+ 250";
    } else
    {
        _expanded = false;
        _propSet.InsertBoolean("expanded", false);
        _springAnimation.InitialValueExpression["FinalValue"] = "this.StartingValue
- 250";
    }
}
```

```
_naviPane.StartAnimation("Offset.X", _springAnimation);
}
```

Now what if you wanted to tie this motion to input? So if the end user swipes out, the panes come out with a Spring motion? More importantly, if the user swipes harder or faster, the motion adapts based on the velocity from the end user.



To do this, you can take our same Spring Animation and pass it into an InertiaModifier with InteractionTracker. For more information about InputAnimations and InteractionTracker, see [Custom manipulation experiences with InteractionTracker](#). We'll assume for this code example, you have already setup your InteractionTracker and VisualInteractionSource. We'll focus on creating the InertiaModifiers that will take in a NaturalMotionAnimation, in this case a spring.

C#

```
// InteractionTracker and the VisualInteractionSource previously setup
// The open and close ScalarSpringAnimations defined earlier
private void SetupInput()
{
    // Define the InertiaModifier to manage the open motion
    var openMotionModifer =
    InteractionTrackerInertiaNaturalMotion.Create(compositor);

    // Condition defines to use open animation if panes in non-expanded view
    // Property set value to track if open or closed is managed in other part
    of code
    openMotionModifer.Condition = _compositor.CreateExpressionAnimation(
    "propset.expanded == false");
    openMotionModifer.Condition.SetReferenceParameter("propSet", _propSet);
}
```

```

openMotionModifer.NaturalMotion = _openSpringAnimation;

// Define the InertiaModifer to manage the close motion
var closeMotionModifier =
InteractionTrackerInertiaNaturalMotion.Create(_compositor);

// Condition defines to use close animation if panes in expanded view
// Property set value to track if open or closed is managed in other part
of code
closeMotionModifier.Condition =
_compositor.CreateExpressionAnimation("propSet.expanded == true");
closeMotionModifier.Condition.SetReferenceParameter("propSet", _propSet);
closeMotionModifier.NaturalMotion = _closeSpringAnimation;

_tracker.ConfigurePositionXInertiaModifiers(new
InteractionTrackerInertiaNaturalMotion[] { openMotionModifer,
closeMotionModifier});

// Take output of InteractionTracker and assign to the pane
var exp = _compositor.CreateExpressionAnimation("-tracker.Position.X");
exp.SetReferenceParameter("tracker", _tracker);
ElementCompositionPreview.GetElementVisual(pageNavigation).
StartAnimation("Translation.X", exp);
}

```

Now you have both a programmatic and input-driven spring animation in your UI!

In summary, the steps to using a spring animation in your app:

1. Create your SpringAnimation off your Compositor.
2. Define properties of the SpringAnimation if you wanted non-default values:
 - DampingRatio
 - Period
 - Final Value
 - Initial Value
 - Initial Velocity
3. Assign to target.
 - If you're animating a CompositionObject property, pass in SpringAnimation as parameter to StartAnimation.
 - If you want to use with input, set NaturalMotion property of an InertiaModifier to SpringAnimation.

Input-driven animations

Article • 10/20/2022

This article provides an introduction to the InputAnimation API, and recommends how to use these types of animations in your UI.

Prerequisites

Here, we assume that you're familiar with the concepts discussed in these articles:

- [Relation based animations](#)

Smooth motion driven from user interactions

In the Fluent Design language, interaction between end users and apps is of the utmost importance. Apps not only have to look the part, but also respond naturally and dynamically to the users that interact with them. This means when a finger is placed on the screen, the UI should gracefully react to changing degrees of input; scrolling should feel smooth, and stick to a finger panning across the screen.

Building UI that dynamically and fluidly responds to user input results in higher user engagement - Motion now not only looks good, but feels good and natural when users interact with your different UI experiences. This enables end users to more easily connect with your application, making the experience more memorable and delightful.

Expanding past just touch

Although touch is one of the more common interfaces end users use to manipulate UI content, they will also use various other input modalities such mouse and pen. In these cases, it is important that end users perceive that your UI responds dynamically to their input, regardless of what input modality they choose to use. You should be cognizant of the different input modalities when designing input-driven motion experiences.

Different Input-Driven Motion Experiences

The InputAnimation space provides several different experiences for you to create dynamically responding motion. Like the rest of the Windows UI Animation system, these input-driven animations operate on an independent thread, which helps contribute to the dynamic motion experience. However, in some cases where the

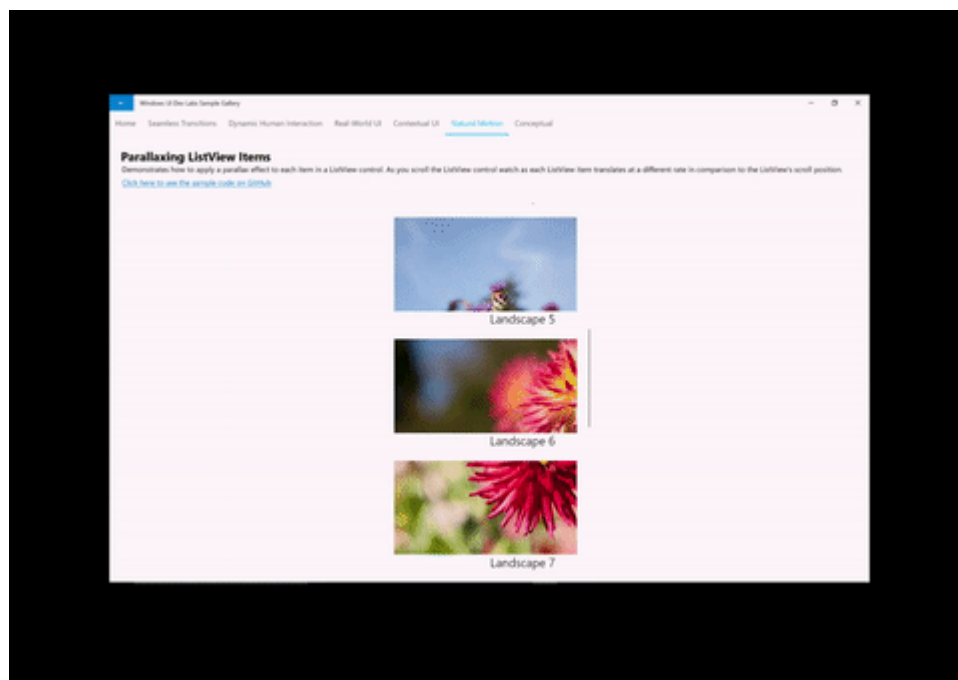
experience leverages existing XAML controls and components, parts of those experiences are still UI thread bound.

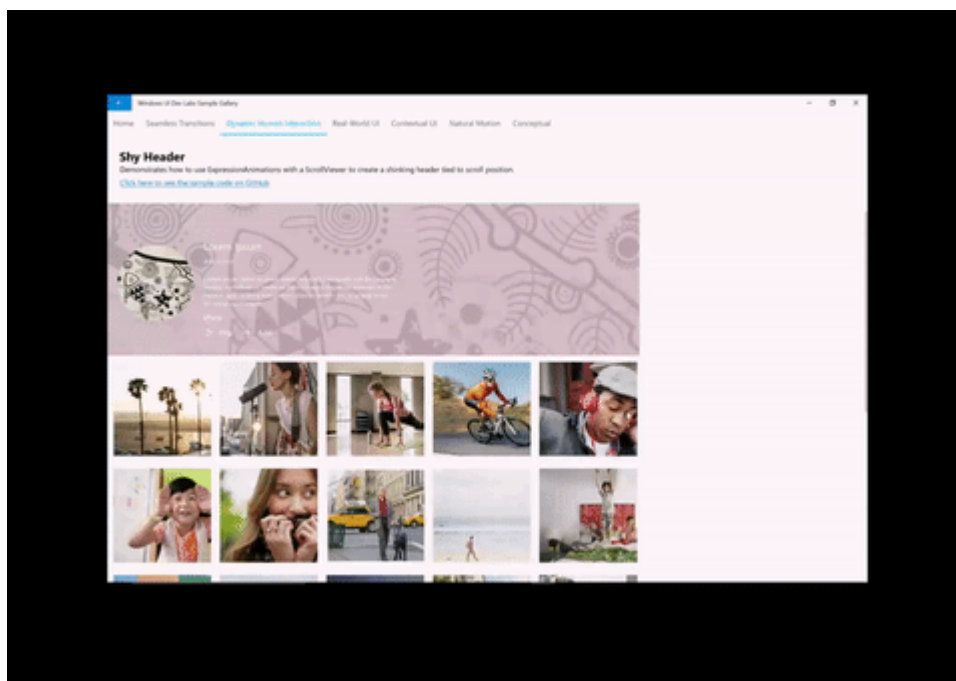
There are three core experiences that you create when building dynamic input-driven motion animations:

1. Enhancing Existing ScrollView Experiences – enable the position of a XAML ScrollView to drive dynamic animation experiences.
2. Pointer Position-driven Experiences – utilize the position of a cursor on a hit tested UIElement to drive dynamic animation experiences.
3. Custom Manipulation experiences with InteractionTracker – create a fully customized, off-thread manipulation experiences with InteractionTracker (such as a scrolling/zooming canvas).

Enhancing Existing ScrollView Experiences

One of the common ways to create more dynamic experiences is to build on top of an existing XAML ScrollView control. In these situations, you leverage the scroll position of a ScrollView to create additional UI components that make a simple scrolling experience more dynamic. Some examples include Sticky/Shy Headers and Parallax.





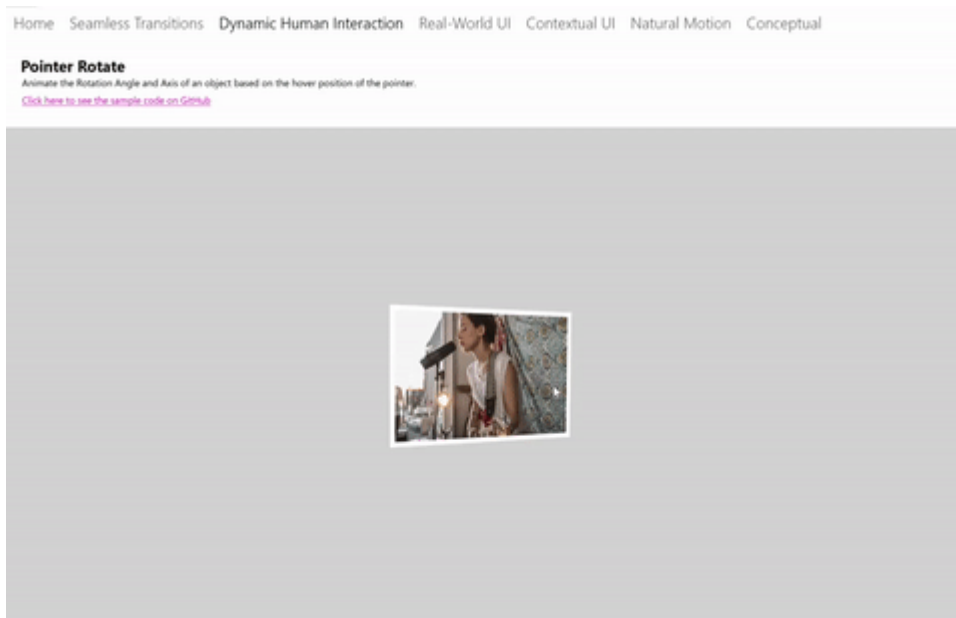
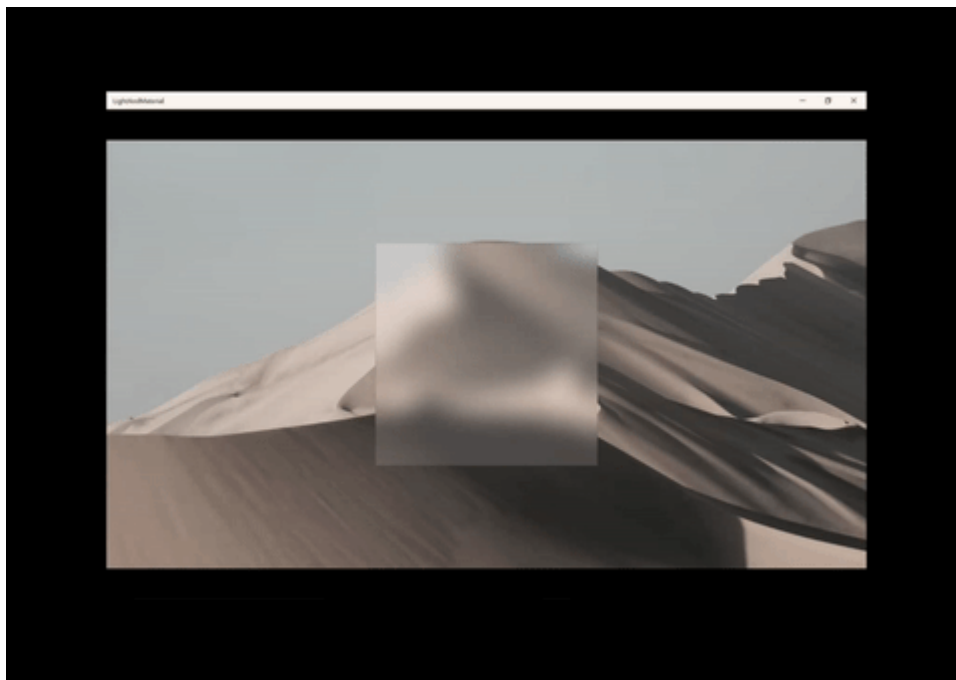
When creating these types of experiences, there is a general formula to follow:

1. Access the `ScrollManipulationPropertySet` off of the XAML `ScrollViewer` you wish to drive an animation.
 - Done via the `ElementCompositionPreview.GetScrollViewerManipulationPropertySet(UIElement element)` API
 - Returns a `CompositionPropertySet` containing a property called **Translation**
2. Create a `Composition ExpressionAnimation` with an equation that references the `Translation` property.
3. Start the animation on a `CompositionObject`'s property.

For more info on building these experiences, see [Enhance existing ScrollViewer experiences](#).

Pointer Position-driven experiences

Another common dynamic experience involving input is to drive an animation based on the position of a pointer such as a Mouse cursor. In these situations, developers leverage the location of a cursor when hit tested within a XAML `UIElement` that makes experiences like Spotlight Reveal possible to create.

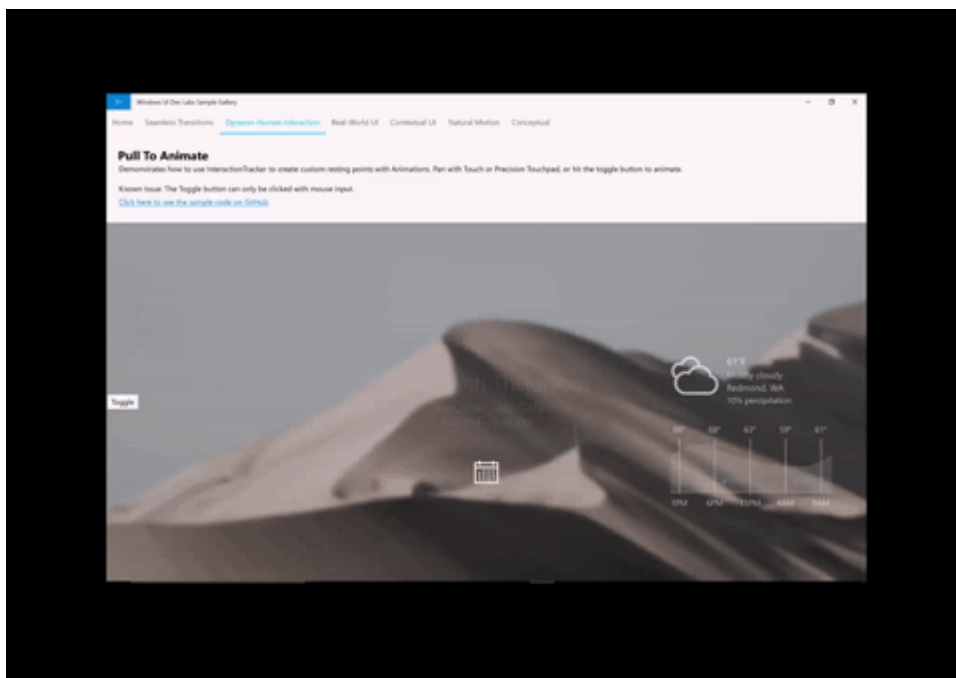
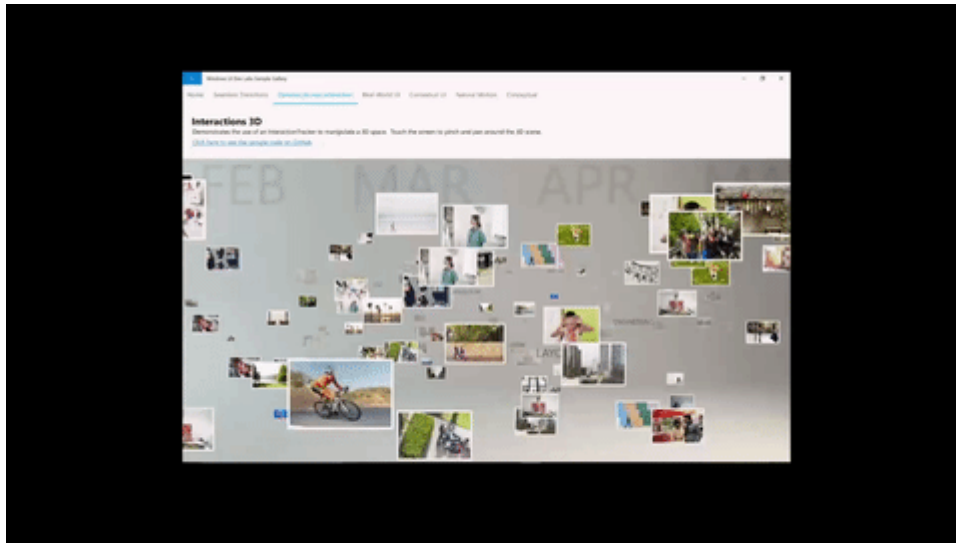


When creating these types of experiences, there is a general formula to follow:

1. Access the `PointerPositionPropertySet` off a XAML `UIElement` that you wish to know the cursor position when hit tested.
 - Done via the `ElementCompositionPreview.GetPointerPositionPropertySet(UIElement element)` API
 - Returns a `CompositionPropertySet` containing a property called **Position**
2. Create a `CompositionExpressionAnimation` with an equation that references the `Position` property.
3. Start the animation on a `CompositionObject`'s property.

Custom manipulation experiences with InteractionTracker

One of the challenges with utilizing a XAML ScrollViewer is that it is bound to the UI thread. As a result, the scrolling and zooming experience can often lag and jitter if the UI thread becomes busy and results in an unappealing experience. In addition, it is not possible to customize many aspects of the ScrollViewer experience. InteractionTracker was created to solve both issues by providing a set of building blocks to create custom manipulation experiences that are run on an independent thread.



When creating experiences with InteractionTracker, there is a general formula to follow:

1. Create your InteractionTracker object and define its properties.
2. Create VisualInteractionSources for any CompositionVisual that should capture input for InteractionTracker to consume.

3. Create a `Composition ExpressionAnimation` with an equation that references the `Position` property of `InteractionTracker`.
4. Start the animation on a `Composition Visual`'s property that you wish to be driven by `InteractionTracker`.

Enhance existing ScrollViewer experiences

Article • 10/20/2022

This article explains how to use a XAML ScrollViewer and ExpressionAnimations to create dynamic input-driven motion experiences.

Prerequisites

Here, we assume that you're familiar with the concepts discussed in these articles:

- [Input-driven animations](#)
- [Relation based animations](#)

Why build on top of ScrollViewer?

Typically, you use the existing XAML ScrollViewer to create a scrollable and zoomable surface for your app content. With the introduction of the Fluent Design language, you should now also be focusing on how to use the act of scrolling or zooming a surface to drive other motion experiences. For example, using scrolling to drive a blur animation of a background or drive the position of a "sticky header".

In these scenarios, you are leveraging the behavior or manipulation experiences like Scrolling and zooming to make other parts of your app more dynamic. These in turn enable the app to feel more cohesive, making the experiences more memorable in the eyes of end users. By making the app UI more memorable, end users will engage with the app more frequently and for longer periods.

What can you build on top of ScrollViewer?

You can leverage the position of a ScrollViewer to build a number of dynamic experiences:

- Parallax – use the position of a ScrollViewer to move background or foreground content at a relative rate to the scroll position.
- StickyHeaders – use the position of a ScrollViewer to animate and "stick" a header to a position
- Input-Driven Effects – use the position of a Scrollviewer to animate a Composition Effect such as Blur.